

JavaScript Basics

Agenda

- Basics of JavaScript
 - Types of JavaScript (Internal & External)
 - Variables
 - Data Types
 - Functions
 - Higher Order Functions (HOF)
-

1. Basics of JavaScript

JavaScript (JS) is a high-level, interpreted, and dynamically typed programming language mainly used for building interactive web applications. It executes on the client side (browser) as well as the server side (Node.js).

2. Types of JavaScript Usage

2.1 Internal JavaScript

JavaScript code written directly inside an HTML document using the `<script>` tag.

```
<script>
  // JavaScript code goes here
  console.log("Hello World");
</script>
```

Use Case: Small scripts, quick testing, learning purposes.

2.2 External JavaScript

JavaScript code written in a separate `.js` file and linked to the HTML document.

```
<script src="myscript.js"></script>
```

Advantages:

- Code reusability
 - Better maintainability
 - Cleaner HTML structure
 - Ability to use predefined JavaScript libraries
-

3. Variables in JavaScript

A variable is a container used to store data values.

Key Points

- Data types are **not** specified during variable declaration.
- JavaScript uses **type inference**.

Variable Declaration Keywords

3.1 **let**

- Block scoped
- Value can be reassigned

```
let num = 100;  
num = 200;
```

3.2 **const**

- Block scoped
- Value **cannot** be reassigned once initialized

```
const salary = 4.5;
```

Examples

```
let str = "test";           // string  
const firstName = 'Steve'; // string  
let canVote = true;         // boolean
```

4. Data Types in JavaScript

JavaScript data types are **inferred automatically**.

Checking Type

```
typeof variableName;
```

Primitive Data Types

4.1 Number

Supports both integer and floating-point numbers.

```
let a = 100;  
let b = 4.5;
```

4.2 String

A sequence of characters enclosed in quotes.

```
let firstName = "Steve";  
let lastName = 'Jobs';
```

4.3 Boolean

Represents logical values.

```
let canVote = true;
```

4.4 Undefined

Declared but not initialized.

```
let x;
```

4.5 Object

Represents complex data structures.

```
let emp = { id: 1, name: "John" };
```

5. Built-in Values

5.1 NaN (Not a Number)

- Type: `number`

```
console.log(parseInt("test")); // NaN
```

5.2 Infinity

Occurs when dividing by zero.

```
let ans = 10 / 0; // Infinity
```

5.3 undefined

No value assigned.

5.4 null

Represents no object.

```
let data = null;  
typeof data; // object
```

6. Template Strings

- Enclosed using backticks ``
- Supports variable interpolation

```
let n1 = 10, n2 = 20;  
let res = n1 + n2;
```

```
console.log(`Addition: ${n1} + ${n2} = ${res}`);
```

7. Functions

A function is a reusable block of statements that performs a specific task.

Syntax

```
function functionName() {  
    // function body  
}
```

Characteristics

- Functions are not type-safe
- Arguments are optional
- Can be called multiple times

Example

```
function add(x, y) {  
    const z = x + y;  
    console.log(`${x} + ${y} = ${z} of type ${typeof z}`);  
}  
  
add(22, 7);  
add("Hello", "World");  
add(10, true);
```

8. Default Parameters

Default values are used when arguments are not passed.

```
function add(x = 0, y, z = 0) {  
    let r = x + y + z;  
    console.log(`add(): ${x} + ${y} + ${z} = ${r}`);  
}  
  
add();
```

```
add(10);  
add(22, 7);  
add(11, 22, 33);
```

9. Function **return**

1. What is **return** in JavaScript?

The **return** statement is used inside a function to:

- Send a value back to the caller
- Immediately terminate the execution of the function

A function can return **any type of value** in JavaScript because JS is dynamically typed.

2. Basic Syntax

```
function functionName(parameters) {  
    // statements  
    return value;  
}
```

When **return** is executed:

- The function stops executing
 - Control goes back to the function call
 - The returned value is available to the caller
-

3. Function Without **return**

If a function does **not** explicitly return a value, JavaScript automatically returns **undefined**.

```
function showMessage() {  
    console.log("Hello World");  
}  
  
let result = showMessage();  
console.log(result); // undefined
```

4. Returning a Value

Example: Returning a Number

```
function add(a, b) {  
  return a + b;  
}  
  
let res = add(10, 20);  
console.log(res); // 30
```

10. Built-in Functions

parseInt()

```
parseInt("123"); // 123  
parseInt("123.45"); // 123  
parseInt("Bond007"); // NaN  
parseInt("007Bond"); // 7
```

parseFloat()

```
parseFloat("3.14"); // 3.14
```

eval()

Evaluates a string expression.

```
eval("2 + 3 * 4"); // 14
```

11. Function Alias

Functions are first-class objects and can be assigned to variables.

```
function add(x, y) {  
  return x + y;  
}
```

```
}  
  
const calc = add;  
calc(20, 10);
```

12. Anonymous Functions

Functions without a name.

```
const subtract = function (x, y) {  
  return x - y;  
};  
  
subtract(22, 7);
```

13. Arrow Functions (ES6)

A concise way to write functions.

Multi-line Arrow Function

```
const add = (a, b) => {  
  const res = a + b;  
  console.log(res);  
};
```

Single Parameter

```
const square = n => n * n;
```

Returning Object

```
const createObject = () => ({ message: "Hello" });
```

13. Higher Order Functions (HOF)

A function that:

- Accepts another function as an argument
- Returns a function

Callback Function

A function passed as an argument to another function.

Example

```
function executer(n1, n2, fn) {  
  const res = fn(n1, n2);  
  console.log('res - ' + res);  
}  
  
const add = (n1, n2) => n1 + n2;  
  
executer(10, 20, add);  
executer(10, 20, (a, b) => a - b);  
executer(10, 20, (a, b) => a * b);
```
